

入門指南

安裝

在 NuGet 安裝對應的資料庫 Client，例如：

```
dotnet add package Microsoft.Data.SqlClient
```

基本設定

在應用程式啟動時呼叫 `FacadeConfiguration.SetConfiguration`，至少需設定 `DbProviderFactory` 和連線字串。

```
using CloudyWing.DatabaseFacade;  
using Microsoft.Data.SqlClient;
```

```
FacadeConfiguration.SetConfiguration(SqlClientFactory.Instance, "your-connection-string");
```

基本查詢

```
using CloudyWing.DatabaseFacade;  
  
using CommandExecutor executor = new CommandExecutor();  
executor.CommandText = "SELECT * FROM Customers WHERE Id IN @Id";  
executor.Parameters.Add("Id", new int[] { 1, 2, 3 });  
DataTable dt = executor.CreateDataTable();
```

基本非同步查詢

```
using CloudyWing.DatabaseFacade;  
  
using CommandExecutor executor = new CommandExecutor();  
executor.CommandText = "SELECT * FROM Customers WHERE Id IN @Id";  
executor.Parameters.Add("Id", new int[] { 1, 2, 3 });  
DataTable dt = await executor.CreateDataTableAsync(cancellationToken);
```

If you need to process data sequentially, use `CreateDataReaderAsync` with `DbDataReader.ReadAsync`. See [Async Operations](#) for more examples.

執行非查詢指令

```
using CommandExecutor executor = new CommandExecutor();
executor.SetCommandText("INSERT INTO Customers (Id, Name) VALUES (@Id, @Name)")
    .Parameters.Add("Id", 1)
    .Add("Name", "小明")
    .GetCommandExecutor()
    .Execute();
```

建立 DbParameters 的方法

基本作法

```
// sql = SELECT * FROM Table WHERE Id = @Id AND Ids IN @Ids

// 正常使用
executor.Parameters.Add("Id", 1);

// 遇到 IN 時可傳入 IEnumerable 型別, 實際會變成 @{Prefix}_Ids_{流水號}
executor.Parameters.Add("Ids", new int[] { 1, 2 });
```

使用 ParameterMetadata

```
// 建立一筆
executor.Parameters.Add(new ParameterMetadata {
    ParameterName = "Id",
    Value = 1
});

// 建立多筆
executor.Parameters.AddRange(param1, param2);
executor.Parameters.AddRange(new ParameterMetadata[] { param1, param2 });
```

使用 IDbDataParameter

```
// 建立一筆
SqlParameter param1 = new SqlParameter("Id", id);
executor.Parameters.Add(param1);

// 建立多筆
executor.Parameters.AddRange(param1, param2);
executor.Parameters.AddRange(new SqlParameter[] { param1, param2 });
```

使用 Dictionary

```
executor.Parameters.AddRange(new Dictionary<string, object> {
    ["Id"] = id,
    ["Name"] = name
});
```

使用 object

```
executor.Parameters.AddRange(new {  
    Id = id,  
    Name = name  
});
```

Method Chaining

```
int result = executor.SetCommandText("INSERT INTO Customers (Id, Name) VALUES (@Id, @Name)")  
    .Parameters.Add("Id", 10).Add("Name", "新增測試").GetCommandExecutor()  
    .Execute();
```

執行資料庫作業 API 後，初始化 Command 資訊

資料庫作業 API

- `IDataReader CreateDataReader(ResetItems thenReset = ResetItems.All, CommandBehavior behavior = CommandBehavior.SequentialAccess)`
- `DataTable CreateDataTable(ResetItems thenReset = ResetItems.All)`
- `object QueryScalar(ResetItems thenReset = ResetItems.All)`
- `int Execute(ResetItems thenReset = ResetItems.All)`

ResetItems

Flags Enum，每個資料庫異動 API 皆有這個參數，當資料庫異動完畢後，會依照這個參數，決定初始化的 Command Info，總共如下：

名稱	初始值	說明
None		不異動
CommandText	Null	
CommandTimeout	DefaultCommandTimeout	
CommandType	CommandType.Text	
Parameters	Clear()	
All		全部初始化

用途

在某些情況下，執行完資料庫作業後，會需要保留原資料，例如有時要新增多筆資料時，CommandText 是相同的，只有 Parameters 需要給予新值。

```
executor.CommandText = "INSERT INTO Customers (Id, Name) VALUES (@Id, @Name)";
executor.Parameters.AddRange(new {
    Id = 1,
    Name = "小明",
});
// 只初始化 Parameters, CommandText仍保留
executor.Execute(ResetItems.Parameters);
executor.Parameters.AddRange(new {
    Id = 2,
    Name = "小王",
});
```

```
});  
executor.Execute(ResetItems.Parameters);
```

Transaction 使用方法

使用 Transaction 時，請將 `KeepConnection` 設為 `true`，可從以下方法設定：

- `FacadeConfiguration.DefaultKeepConnection`
- `CommandExecutor` 建構子參數

如果是在 `async / await` 流程中使用，Transaction 的建立方式不變，只需要把 `CreateDataTable`、`QueryScalar`、`Execute` 等 API 換成對應的 Async 版本即可。

範例

```
using (CommandExecutor executor = new CommandExecutor(true))
using (IDbTransaction tran = executor.BeginTransaction()) {
    executor.CommandText = "INSERT INTO Customers (Id, Name) VALUES (@Id, @Name)";

    executor.Parameters.AddRange(new {
        Id = 1,
        Name = "小明"
    });
    executor.Execute(ResetItems.Parameters);

    executor.Parameters.AddRange(new {
        Id = 2,
        Name = "小王"
    });
    executor.Execute();

    tran.Commit();
}
```

非同步操作

DatabaseFacade 現在提供對應的非同步 API，讓原本偏向同步流程的用法，也能自然地整合到 `async / await` 為主的應用程式中。

可用方法

同步方法	非同步方法	回傳型別
CreateDataReader	CreateDataReaderAsync	Task<DbDataReader>
CreateDataTable	CreateDataTableAsync	Task<DataTable>
QueryScalar	QueryScalarAsync	Task<object?>
Execute	ExecuteAsync	Task<int>

所有非同步方法都會保留原本的同步 API，並額外提供 `CancellationToken` 參數。

非同步查詢 DataTable

```
using CommandExecutor executor = new CommandExecutor();
executor.CommandText = "SELECT * FROM Customers WHERE Id IN @Ids";
executor.Parameters.Add("Ids", new int[] { 1, 2, 3 });

DataTable dataTable = await executor.CreateDataTableAsync(cancellationToken);
```

非同步逐筆讀取資料

如果資料量較大，或想要邊讀邊處理，可以直接使用 `CreateDataReaderAsync`：

```
using CommandExecutor executor = new CommandExecutor();
executor.CommandText = "SELECT Id, Name FROM Customers WHERE Id IN @Ids";
executor.Parameters.Add("Ids", new int[] { 1, 2, 3 });

using DbDataReader reader = await executor.CreateDataReaderAsync(cancellationToken:
cancellationToken);
while (await reader.ReadAsync(cancellationToken)) {
    Console.WriteLine($"{reader["Id"]}, {reader["Name"]}");
}
```

非同步執行單一值查詢與非查詢命令


```

using CommandExecutor executor = new CommandExecutor();

long count = Convert.ToInt64(await executor
    .SetCommandText("SELECT COUNT(1) FROM Customers")
    .QueryScalarAsync(cancellationToken: cancellationToken));

int affectedRows = await executor
    .SetCommandText("UPDATE Customers SET Name = @Name WHERE Id = @Id")
    .Parameters.Add("Name", "小明")
    .Add("Id", 1)
    .GetCommandExecutor()
    .ExecuteAsync(cancellationToken: cancellationToken);

```

與 Transaction 一起使用

Transaction 的開啟方式不變，仍然透過 `BeginTransaction` 進行；只要將實際執行命令的部分改成 Async 版本即可。

```

using CommandExecutor executor = new CommandExecutor(true);
using IDbTransaction transaction = executor.BeginTransaction();

await executor
    .SetCommandText("INSERT INTO Customers (Id, Name) VALUES (@Id, @Name)")
    .Parameters.Add("Id", 1)
    .Add("Name", "小明")
    .GetCommandExecutor()
    .ExecuteAsync(ResetItems.Parameters, cancellationToken);

await executor
    .SetCommandText("INSERT INTO Customers (Id, Name) VALUES (@Id, @Name)")
    .Parameters.Add("Id", 2)
    .Add("Name", "小王")
    .GetCommandExecutor()
    .ExecuteAsync(cancellationToken: cancellationToken);

transaction.Commit();

```

⊗ IMPORTANT

若要使用 `BeginTransaction`，請記得將 `KeepConnection` 設為 `true`，否則會拋出 `KeepConnectionRequiredException`。

建立 SQL 語法 Log

SQL Server 範例

```
FacadeConfiguration.OnCommandCreated = cmd => {
    string sql = cmd.CommandText;
    foreach (IDataParameter param in cmd.Parameters) {
        string value;
        switch (param.Value) {
            case DateTime dateTime:
                value = $"{dateTime:yyyy/MM/dd HH:mm:ss}";
                break;
            case long:
            case int:
            case short:
            case byte:
            case ulong:
            case uint:
            case ushort:
            case sbyte:
            case float:
            case double:
            case decimal:
                value = param.ToString();
                break;
            case DBNull:
            case null:
                value = "NULL";
                break;
            default:
                value = $"{param.Value?.ToString()}";
                break;
        }

        string pattern = string.Format("{0}(?=[\\W])|{0}$", "@" + param.ParameterName);
        sql = Regex.Replace(sql, pattern, value, RegexOptions.IgnoreCase);
    }

    Console.WriteLine(sql);
};
```

Oracle 範例

```

FacadeConfiguration.OnCommandCreated = cmd => {
    string sql = cmd.CommandText;
    foreach (IDbDataParameter param in cmd.Parameters) {
        string value;
        if (param.Value is null || param.Value is DBNull) {
            value = "NULL";
        } else if (param.Value is DateTime dateTime) {
            value = string.Format(
                "TO_DATE('{0}', '{1}']",
                dateTime.ToString("yyyy/MM/dd HH:mm:ss"),
                "yyyy/mm/dd hh24:mi:ss"
            );
        } else if (param.Value is long || param.Value is int
            || param.Value is short || param.Value is byte
            || param.Value is ulong || param.Value is uint
            || param.Value is ushort || param.Value is sbyte
            || param.Value is float || param.Value is double || param.Value is decimal) {
            value = param.Value.ToString();
        } else {
            value = "'" + param.Value.ToString() + "'";
        }

        string pattern = string.Format("{0}(\\W)]|{0}$", ":" + param.ParameterName);
        sql = Regex.Replace(sql, pattern, value, RegexOptions.IgnoreCase);
    }

    Console.WriteLine(sql);
};

```

注意事項

1. 上述將 DbCommand 轉換成可執行的 SQL 語法作法僅供參考，有一些特別型別可能無法處理，再自行調整。
2. `Console.WriteLine(sql)`；請自行替換成專案實際寫入 Log 的語法。